

CHRONIX EDU

Architecture & Build Guide

Version 1.2 | Updated June 2026 | Chronix Technology Limited
Incorporates: pre-build session decisions, multi-tenancy isolation model

v1.2
CHANGES

This version adds: (1) title and teacher_mode fields on users table, (2) default seeding for school_settings on creation, (3) Multi-Tenancy Security section with RLS enforcement rules and service role key prohibition, (4) updated schema notes for Chronix Edu, (5) font stack updated to Poppins/Inter.

1. Executive Summary

This document provides the full architecture and build guidance for Chronix Edu — a multi-school, multi-role School Management System (SMS) built as a SaaS product by Chronix Technology Limited. It is designed to serve School Administrators, Principals, Teachers, Parents, Students, and Registrars — each with tailored dashboards and role-specific workflows.

GOAL

Build a scalable, configurable, multi-tenant SaaS product that can be sold to multiple schools — not a single-school application. First client: Child Prime Onyx School.
Domain: edu.chronixtechnology.com.

2. Multi-Tenancy Architecture & Data Isolation

Multiple schools use the same platform. Every data breach between schools is catastrophic — both legally and reputationally. This section defines the isolation model, the rules that make it secure, and the tests that prove it works.

2.1 Chosen Model: Shared Schema + Row Level Security

Chronix Edu uses Option 1: a single PostgreSQL database with school_id on every tenant-scoped table, enforced by Supabase Row Level Security (RLS). This is the correct model for MVP through the first 50+ schools.

Model	How It Works	Decision
Database per school	Each school gets its own Supabase project	SKIP — cost multiplies per school, migrations across N projects, no cross-school analytics
Schema per school	Separate schema in one DB — school_a.students, school_b.students	SKIP — migrations must run per schema, Supabase RLS bypassed, complex API routing
Shared schema + school_id (RLS)	All schools share tables, filtered by school_id. RLS enforces isolation.	USE THIS — fastest to build, easiest to scale, correct for hundreds of schools

2.2 The Four Rules That Make Option 1 Secure

Option 1 is not weak. Notion, Linear, and most serious SaaS products use it. The risk is not the architecture — it is sloppy implementation. These four rules are non-negotiable:

#	Rule	Detail
1	school_id from JWT only	The API must never trust req.body.school_id or a URL parameter to identify the school. It reads school_id exclusively from the verified JWT token. A school cannot impersonate another by modifying a request.
2	RLS on every table — no exceptions	Not most tables. Every table. A single table without RLS policies is a full breach. This includes junction tables (parent_students, teacher_assignments) and audit_logs.
3	Service role key NEVER in application code	The Supabase service role key bypasses RLS entirely. It is used ONLY in server-side migration scripts. It must never appear in any API request handler, middleware, or client-side code. Exposure of this key = complete breach of all schools.
4	T3 isolation test runs before every deployment	Cross-tenant isolation is not a one-time test. It must run in CI on every push. A regression here is catastrophic.

CRITICAL	The service role key is the master key to every school's data. Store it only in server-side environment variables. Never log it. Never send it in an API response. Never put it in .env.local on a machine that others access.
----------	--

2.3 RLS Policy Structure

Every tenant-scoped table requires these two RLS policies. Replace [table_name] with the actual table.

Policy	SQL Pattern
Isolation policy	CREATE POLICY school_isolation ON [table_name] USING (school_id = (auth.jwt()->>'school_id')::uuid);
Enable RLS	ALTER TABLE [table_name] ENABLE ROW LEVEL SECURITY;
Super admin bypass	Use service role key in migration scripts only — not in API handlers.

2.4 Migration to Per-Schema When You Need It

Option 1 handles hundreds of schools comfortably. When Chronix Edu reaches 50+ schools and performance profiling shows isolation-related bottlenecks, migrate to per-schema isolation. That migration is a well-understood operation and does not require rebuilding the application — only adding a schema-routing layer. Do not pre-optimize for this now.

3. Technology Stack

3.1 Locked Stack — No Substitutions

Layer	Technology	Reason
Frontend	Next.js 14 + Tailwind CSS	SSR, great developer experience, easy role-based routing
Backend API	Node.js + Express (TypeScript)	Familiar, large ecosystem, easy REST API
Primary Database	PostgreSQL via Supabase	Relational, supports RLS; Supabase adds auth + storage
Auth	Supabase Auth + JWT	Role-based JWT with custom claims
File Storage	Supabase Storage	Report cards, photos, assignments
PDF Generation	Puppeteer	Server-side report card generation
Offline	Dexie.js (IndexedDB)	Attendance and score entry offline queue
Email	SendGrid	Transactional email at launch
SMS	Termii (Phase 3)	Nigerian SMS gateway, per-school API key
Payments	Paystack (Phase 3)	Best Nigerian payment gateway
Hosting — Web	Vercel	CI/CD, scales well, Next.js native
Hosting — API	Railway	Express API hosting
Forms	React Hook Form	All forms throughout the system
Charts	Recharts (Phase 3+)	Analytics dashboards

3.2 Font Stack

Usage	Font	Weight
Headings	Poppins	600 (SemiBold)
Body / UI text	Inter	400 (Regular)
Code snippets	Courier New / monospace	400

4. Database Schema

4.1 Schema Design Rules

- UUID primary keys on all tables.
- school_id UUID FK on every tenant-scoped table — no exceptions.
- RLS enabled on every table before any data is inserted.
- All academic data (scores, attendance, results) carries both session_id and term_id.
- Soft-delete preferred over hard-delete — set is_active=false rather than DELETE.
- audit_logs captures every write to sensitive tables.

4.2 Core Table Definitions

users table — updated fields for Chronix Edu

Column	Type	Constraint	Notes
id	UUID	PK, default gen_random_uuid()	
school_id	UUID	FK to schools, NULLABLE	Null for super_admin only
email	VARCHAR(255)	UNIQUE, NOT NULL	
password_hash	TEXT	NOT NULL	Bcrypt hash
role	ENUM	NOT NULL	super_admin, principal, teacher, parent, student, registrar, bursar
first_name	VARCHAR(100)	NOT NULL	
last_name	VARCHAR(100)	NOT NULL	
title	VARCHAR(20)	NULLABLE	NEW: Dr., Prof., Engr., Mrs., Mr., Chief — used in greetings
teacher_mode	ENUM('class','subject')	NOT NULL DEFAULT 'subject'	NEW: class=primary teacher, subject=secondary teacher. Set at creation, not editable by teacher.
phone	VARCHAR(20)	NULLABLE	
is_active	BOOLEAN	NOT NULL DEFAULT true	Deactivate instead of delete
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

IMPORTANT

The title and teacher_mode columns were added during the Chronix Edu pre-build dashboard review session. teacher_mode is set by the admin at user creation and cannot be changed by the teacher themselves. title is used in all dashboard greetings: 'Good morning, Dr. Okafor' — never first name only.

school_settings — seeded on school creation

When a school is created (POST /api/schools), the system auto-creates a school_settings row with Nigerian defaults. These values are configurable but must be present from day one so score entry and grading work immediately.

Setting	Default Value	Notes
Grading scale	A=70-100, B=60-69, C=50-59, D=40-49, F=0-39, cutoff=40	Stored in academic_config JSONB. Configurable per school.
Assessment components	CA1 10%, CA2 10%, Mid-Term 10%, Exam 70% (sum=100)	Stored in academic_config JSONB. Weights must sum to 100.
Academic calendar	3 terms: First Term (Sep-Dec), Second Term (Jan-Apr), Third Term (May-Jul)	Stored in academic_config JSONB. Configurable per school.

All 16 core tables — summary

Table	Key Fields & Notes
schools	id, name, slug, logo_url, stamp_url, address, phone, email, primary_colour, subscription_tier, is_active
school_settings	school_id UNIQUE, identity_config JSONB, academic_config JSONB, report_config JSONB, notification_config JSONB
academic_sessions	id, school_id, name, start_date, end_date, is_current. Partial unique index: one is_current=true per school.
terms	id, session_id FK, school_id, name (First/Second/Third), start_date, end_date, is_current
classes	id, school_id, name, level, stream. Unique name per school.
subjects	id, school_id, name, code, is_active. Unique code per school.
users	id, school_id NULLABLE, email, password_hash, role ENUM, first_name, last_name, title VARCHAR(20), teacher_mode ENUM('class','subject'), phone, is_active
students	id, school_id, user_id FK, admission_no, dob, gender, address, photo_url, blood_group, emergency_contact_name, emergency_contact_phone
parent_students	id, parent_id FK users, student_id FK students, relationship_type, is_primary_contact
student_classes	id, student_id, class_id, session_id, enrolled_at. History preserved on promotion — never delete old rows.
teacher_assignments	id, teacher_id FK users, class_id, subject_id, term_id, school_id
assessment_configs	id, school_id, term_id, subject_id NULLABLE, class_level NULLABLE, is_default BOOLEAN
assessment_components	id, config_id FK, name, max_score, weight_percent, display_order. CHECK: weights per config sum to 100.
scores	id, school_id, student_id, subject_id, term_id, component_id, score DECIMAL, entered_by FK users, entered_at, updated_at
result_status	id, student_id, term_id, school_id, status ENUM('draft','submitted','approved','published'), updated_by, updated_at
audit_logs	id, school_id, user_id, action_type, entity, entity_id, old_value JSONB,

	new_value JSONB, ip_address, created_at. No DELETE allowed.
--	---

5. Result Approval Workflow

In Nigerian schools, results do not go from teacher entry directly to parent visibility. There is a trust and verification chain. The system enforces this strictly.

Stage	Status	What Happens
1	DRAFT	Teacher enters CA scores, test scores, exam scores. Editable.
2	SUBMITTED	Teacher confirms subject complete. Locks score editing. Triggers review.
3	APPROVED	Principal verifies all subjects submitted. Cannot approve if any are draft.
4	PUBLISHED	Results visible to parents and students. Report cards downloadable. Parent notification queued.
RETURN	Back to DRAFT	Principal can return results to teacher with a reason. Teacher notified.

RULE	<code>validateStatusTransition(current, next)</code> must guard every endpoint. Transitions are one-directional except for the explicit return endpoint. The guard makes invalid transitions (e.g. draft → published) impossible — not just undesirable.
------	--

6. User Roles & Permissions

SECURITY RULE

Never enforce permissions only on the frontend. A user can bypass the UI and call the API directly. All permission checks must live in backend middleware. `requireRole()` is mandatory on every sensitive endpoint.

Permission	Super Admin	Principal	Teacher	Parent	Student	Registrar
Manage schools/tenants	✓	✗	✗	✗	✗	✗
Configure school settings	✓	✓	✗	✗	✗	✗
Approve & publish results	✓	✓	✗	✗	✗	✗
Enter subject scores	✓	✗	✓	✗	✗	✗
View all students	✓	✓	Own class	✗	Self	✓
View results	✓	✓	Own class	Own ward	Self	✓
Mark attendance	✓	✓	✓	✗	✗	✗
Manage student enrollment	✓	✗	✗	✗	✗	✓
Generate report cards	✓	✓	✗	Download	Download	✓
Manage fees/billing	✓	✗	✗	View balance	✗	✗

7. UI Decisions — Locked

These decisions were confirmed during the Chronix Edu dashboard prototype review session. They are locked — do not change them without explicit approval.

Decision	What to Build
User greeting format	Use title if available: 'Good morning, Dr. Okafor'. Never first name only. Fall back to full name if no title stored.
Principal result grid	Dropdown class selector — not horizontal scroll. One class at a time.
Teacher score entry mode	Mode locked per account (teacher_mode). Set by admin at user creation. No toggle shown to teacher.
Score submission granularity	Per class per subject. Never bulk. Button text: 'Submit JSS 2A for approval'.
Parent child selector	Hidden if parent has only one child. Dropdown chevron if two or more.
Parent component breakdown	Full breakdown: CA1, CA2, Mid-Term, Exam shown individually per subject.
Parent fee card	Always visible from launch. Pay Now disabled with 'Coming soon' until Phase 3.
Parent bottom navigation	5 tabs always visible: Home, Results, Attendance, Messages, Fees.
Attendance visibility	Always visible to parents regardless of result publication status.
Teacher assignment overview	Assignment cards above score grid on same page — not a separate page.

8. Assessment Configuration Engine

This engine allows each school to define how assessments are structured and weighted. It supports four resolution levels so schools can have different configs for different subjects or class levels.

8.1 Resolution Priority (highest to lowest)

- 1. Exact subject + class_level match
- 2. Subject-only match
- 3. Class_level-only match
- 4. School-wide default (is_default=true)

RULE

If no config is found at any level, the API returns 400 with a clear message: 'This school has no assessment configuration set up.' Never silently fall back to hardcoded weights.

8.2 Locking Rules

- Once scores have been entered for a term, the assessment config for that term is LOCKED (423 response).
- Locked configs show a padlock icon in the UI. All inputs are disabled.
- To change weights mid-term: create a NEW config for the next term. Never modify a locked one.
- Grading scale changes while published results exist: allowed with a warnings array in the response — not blocked.

9. Security, Compliance & Audit

9.1 JWT Claims

Every JWT must include these custom claims. The API reads `school_id` from the token — never from request parameters.

Claim	Type	Value
<code>user_id</code>	UUID	The user's id from the users table
<code>school_id</code>	UUID	The school this user belongs to (null for <code>super_admin</code>)
<code>role</code>	ENUM string	One of: <code>super_admin</code> , <code>principal</code> , <code>teacher</code> , <code>parent</code> , <code>student</code> , <code>registrar</code> , <code>bursar</code>
<code>email</code>	string	User's email address
<code>title</code>	string or null	Dr., Prof., Engr., Mrs., Mr., Chief — used in dashboard greetings

9.2 What the Audit Log Captures

- Every score entry — `old_value` and `new_value` captured before `upsert`.
- Every result status transition — who approved, who published, when.
- Every school settings change — old and new JSONB values.
- Session and term activations.
- Student class corrections and promotions.
- Behaviour incidents.
- `audit_logs` has no `DELETE` policy — records are permanent.

9.3 Error Response Standard

All API errors must return structured JSON — never HTML, never stack traces in production.

HTTP Code	When to Use
400	Validation failure — missing field, invalid value, weight sum != 100
401	No token or invalid token
403	Valid token but wrong role, or wrong school
409	Conflict — duplicate, referenced record exists
423	Locked — config locked, result locked for editing
500	Unexpected server error — never expose stack trace

10. Development Phases

For Chronix Edu — Target: September 7, 2026

Phase	What Gets Built	Prompts	Duration
Foundation	Database, auth, RLS, multi-tenancy, project setup	P1–P8	Weeks 1–3
Phase 1 — MVP	Students, scoring, result workflow, report cards, dashboards	P9–P20b	Weeks 4–10
Phase 2 — Operations	Attendance, parent portal, assignments, behaviour, messaging, PWA	P21–P31	Weeks 11–13
Phase 3 — Stretch	Fees, Paystack, SMS, analytics, timetable (if time allows)	P32–P40	Post Sep 7

RULE	No task is DONE until its matching test in 'The Test' document passes. Run the test before moving to the next task. T3 (cross-tenant isolation) must pass with 100% certainty before any other task proceeds.
------	---